



FCIV

Drone Control System

Project Report — Flight Controller and Power Distribution Board

David Olson, Sterling Schurr, Corban Critchfield, Ben Jensen

Prepared for Hill Air Force Base

By the Department of Electrical and Computer Engineering

Brigham Young University – Idaho

Contents

Objectives.....	3
Mission.....	3
Scope of Work	3
Flight Controller IV Initiative.....	3
Power Distribution Board (PDB).....	4
Hardware Description.....	4
Specifications.....	4
Schematics.....	5
Assembly Instructions.....	6
Flight Controller IV.....	7
Hardware Description.....	7
Software (PX4).....	8
Using Linux.....	8
Our progress with PX4.....	8
The PX4 Bootloader.....	8
The PX4 Application.....	9
How to load PX4.....	9
Why PX4 still doesn't work for us.....	10
QGroundControl.....	11
PX4IO Coprocessor.....	11
Disable VBUS Sense.....	11
STM32F777 Overdrive Mode.....	11
Recommendations.....	12
Engineering Standards (ABET).....	12
Design to Meet Specific Needs (ABET).....	12
Appendix A: PX4 Repository Modifications for using FCII.....	12
PX4-Bootloader.....	12
PX4-Autopilot.....	14

Objectives

Mission

Design and implement a compact, modular drone flight controller and power distribution system that enables robust flight control, reliable sensor data logging, and streamlined integration of essential components for educational and developmental UAV platforms.

Scope of Work

This project involves the design and development of a custom PCB stack consisting of a flight controller and power distribution board (PDB) for small drones. The system integrates a microcontroller, motion sensors, GPS, data logging via microSD, and PWM outputs for electronic speed controllers (ESCs). The work includes:

- Schematic and PCB layout design using KiCad
- Software development and sensor interfacing
- Mechanical and stack configuration

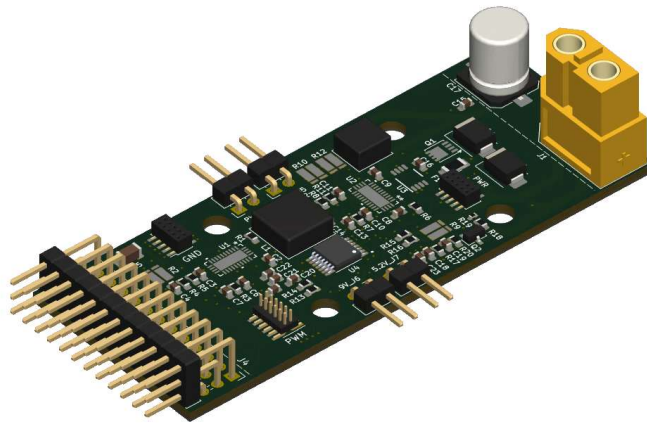
The Drone Flight Controller project should also ensure that the controller supports the drone platform identified in the Autonomous Drone Navigation project.

Flight Controller IV Initiative

The Flight Controller IV initiative is to carry out the flight controller project to completion. This involved developing and manufacturing a PDB, narrowing the project scope, and troubleshooting software.

Another objective of this project defined by Hill Air Force Base is to have the hardware completely sourced in the USA, or allied nations. Previous teams had put a huge emphasis on this objective and as a result the project stagnated with dysfunctional flight controller boards. The Flight Controller IV team made a conscious decision to ignore this objective until a reliable and functional flight controller was guaranteed. Our project focused on rapid prototype and sourcing from convenient and well-established sources, including JLCPCB® and LCSC®, despite the nationality of these companies. We encourage future teams to return to this objective as soon as the product becomes viable. This objective is important and must be implemented in the final design, however, for the sake of progress we warn not to place too much emphasis on this objective early on.

Power Distribution Board (PDB)



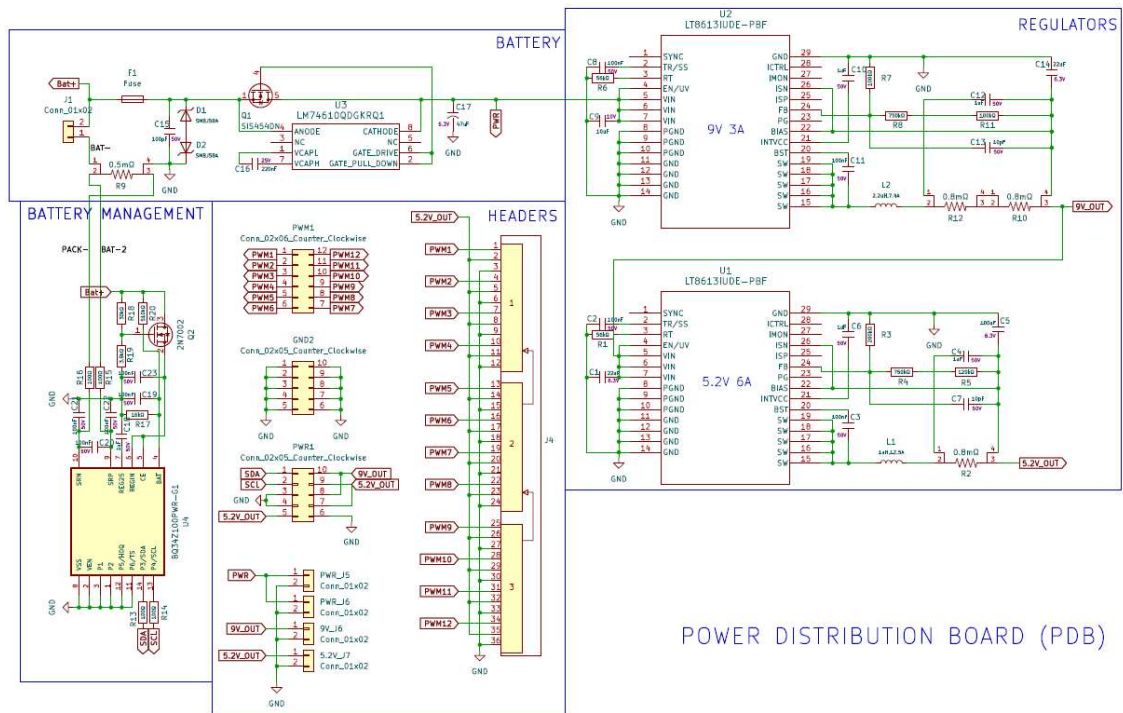
Hardware Description

This board's intent is to supply power to the FC and peripherals; it takes power from the XT60 Battery connector and regulates it to 9V and 5V. The PWM signals are routed from the FC's IC. Various peripherals of the FC take either the 9V line or 5V line, whereas the PWM signals will drive off-board ESCs. There is an integrated BQ34Z100-G1 fuel-gauge IC to measure the remaining battery capacity and state of charge. The fuel-gauge uses I2C to communicate with the FC.

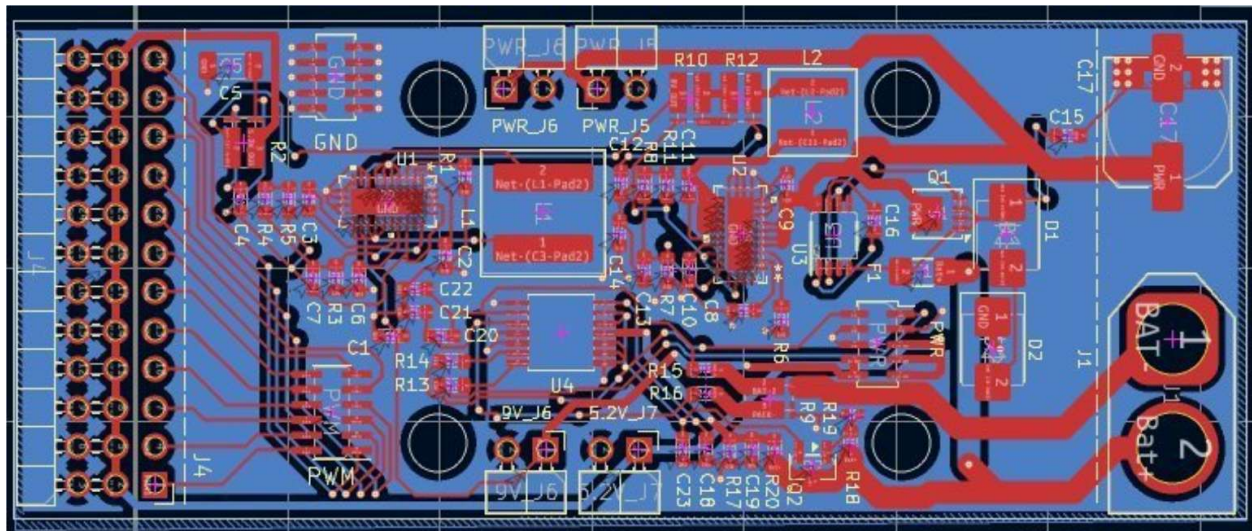
Specifications

- Input voltage: 12-30V
- Input current: 6A max
- Total PWM outputs: 12

Schematics



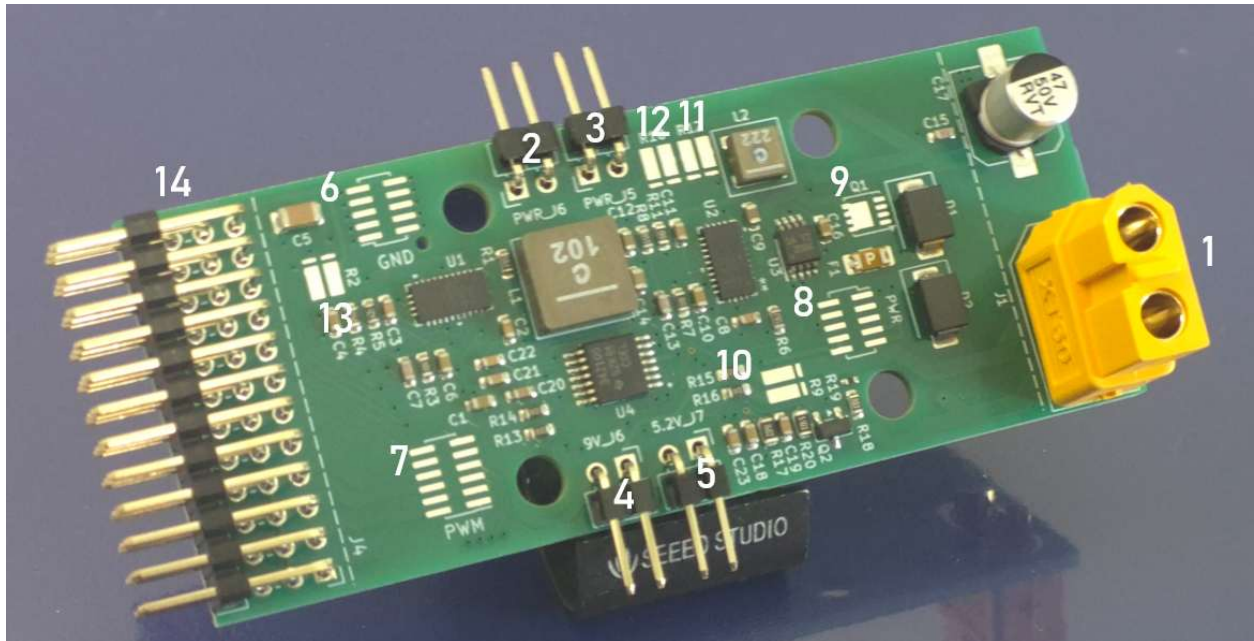
Layout



This design of the PDB layout is more streamlined than previous versions and removes the integrated ESCs to avoid expensive thick copper traces and improved current-carrying capacity to ESCs at lower risk. The ESCs have the potential to draw up to 60A continuous current, and we were concerned this would fry the board. As an alternative,

the current supply to ESCs are to be through a custom XT60 1-5 splitter connector, while control signals are wired from the triple-pin headers. This connector has not been assembled yet.

Assembly Instructions



To enable voltage regulation, the power distribution board must be fully assembled. Many components are pre-placed during manufacturing, but a few components were ordered separately through DigiKey®. These components are listed in the materials section below.

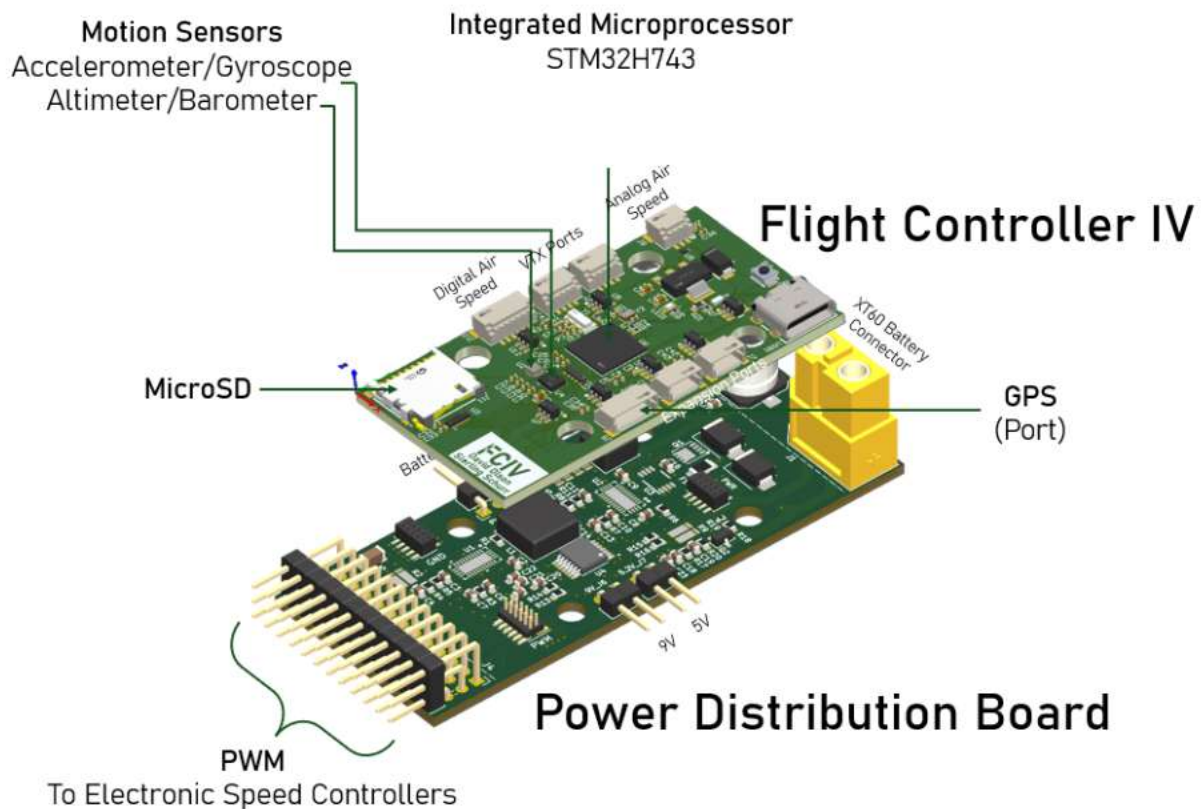
Materials

1. XT60 **MALE** CONNECTOR. Important: the component shown in the image above is the incorrect side of the connector (female, not male). Use the opposite connector to ensure compatibility with common battery cables.
2. 2.54mm pitch 2-pin horizontal header.
3. 2.54mm pitch 2-pin horizontal header.
4. 2.54mm pitch 2-pin horizontal header.
5. 2.54mm pitch 2-pin horizontal header.
6. 1mm pitch 10-pin socket.
7. 1mm pitch 12-pin header.
8. 1mm pitch 10-pin socket.
9. NMOS: SIS454DN-T1-GE3.
10. 0.5mΩ current sense resistor.
11. 0.8mΩ current sense resistor.

12. 0.8mΩ current sense resistor.
13. 0.8mΩ current sense resistor.
14. 2.54mm pitch 12x3-pin horizontal header.

Flight Controller IV

Hardware Description



Flight Controller IV (FCIV) is an expandable flight controller for small- to medium-sized drones. Although based on a vertical take-off and landing (VTOL) flight controller, it can support a variety of airframes.

FCIV stacks on a power distribution board (PDB) which routes pulse-width modulation (PWM) channels to conventional electronic speed controller (ESC) ports and uses step-down switching regulators to supply power to the device.

FCIV includes expansion ports for GPS, analog and digital air-speed sensors, dual video transmission, and two unreserved ports (recommended for ELRS receiver). Integrated devices include an accelerometer, altimeter, and microSD.

For more information, see [FCIV-ProductInformation.docx](#).

Software (PX4)

This flight controller board has been built with the goal of loading PX4 onto it. PX4 is an open-source autopilot firmware that handles the functions needed to accept commands and fly the drone to fulfill those commands. PX4 has robust documentation that can be found on their website at docs.px4.io. In particular, [this page](#) was a good place to start. Our experiences are documented below, but the processes may look different for newer and better supported processors and hardware. Look at the official documentation for authoritative information.

Using Linux

The first thing to note is that PX4 requires Linux to build. Officially, Ubuntu is the recommended distro, although others would probably also work. We were able to use WSL (Windows Subsystem for Linux) to build the code but loading it onto the board was hard because WSL has limited access to hardware. We eventually installed Ubuntu as a dual-boot system on one of our laptops to overcome this. It's not as hard as it sounds, reversible, and is the way we recommend doing it. There are hundreds of tutorials available online to do this.

Our progress with PX4

Our work in PX4 happened largely on Flight Controller II, the board with the STM32F777. We didn't use FCIII because the F722 doesn't have enough storage to hold PX4. We were able to get the PX4 bootloader and the PX4 application loaded onto the board and operating. Unfortunately, due to issues with the integration of peripherals, we couldn't achieve a functioning controller. Near the end of the semester, we received the FCIV boards and attempted to load PX4 on one. We made some progress and were able to get the bootloader loaded but didn't have time to try anything more.

The PX4 Bootloader

The system is made up of a chain of software handing off to each other. First, the STM bootloader has control. Its only job is to start the board and launch the code you have loaded. The code that it launches will be the PX4 bootloader. The PX4 bootloader sets up the board for PX4, then launches the PX4 application, which is what actually runs when the drone is flying.

One important function of the PX4 bootloader is to accept PX4 application builds. The process is first to flash the bootloader to controller memory. Then, when the board boots, the PX4 bootloader will be able to accept a new build of the PX4 app via the toolchain provided by the PX4 repository. The steps on how to do all of this are detailed later in this section.

The bootloader was a separate repository on GitHub (PX4-Bootloader) for older chips such as the F777. For newer chips such as the one on FCIV, the bootloader code was integrated into the main PX4-Autopilot repository.

The PX4 Application

When the board has been loaded with a PX4 application, the bootloader automatically launches it after a defined delay. This application is where all the functions like RC control, PWM output, reading GPS signals etc. happen. Once the board is running this, it can accept commands from the operator, ground control station, or companion computer. The application is loaded via the PX4 bootloader as described later in this section.

How to load PX4

IMPORTANT: The build process for this is different between processors. For the F777, the bootloader is a separate GitHub repo. On newer H7 chips like the one used on FCIV, the bootloader is integrated into the main repo.

IMPORTANT: Some of the code we made will mistakenly reference FCII as FCIII. This was a typo that got propagated through and we don't have time to fix it. Any mention of FCIII in any of the PX4 code is actually talking about FCII. As said before FCIII can't even run PX4.

To get PX4 on FCII, with the F777:

1. Clone the PX4-Bootloader repo
2. Copy our code into the files of PX4-Bootloader as seen in Appendix A of this document. This will make a new bootloader build target
3. cd into PX4-Bootloader folder and run the command "make byui_fciii_bl"
 - a. This will create a .bin file for our board in PX4-Bootloader/build
4. Plug in the board with the BOOT-0 button held down to get into the STM32 bootloader and run "lsusb" on your host computer to see it's there
5. Use dfu-util to flash the created .bin file to the board using this command: "dfu-util -a 0 --dfuse-address 0x08000000 -D build/<target>/<target>.bin"
 - a. 0x08000000 is the memory address that the STM bootloader launches code from. This will place the bootloader there.
6. Once flashed, un-plug and re-plug the board's USB and then run "lsusb" again. If successful, the board should show up.
7. Clone the PX4-Autopilot repo
8. Copy our code into the files of PX4-Autopilot as seen in Appendix A of this document. This will make a new PX4 build target
9. cd into the PX4-Autopilot folder and run "make byui_fciii_default upload"

- a. Similar to before, this will create a .bin file for our board in /builds
 - b. The “upload” option will launch the built-in tool for uploading to the board. When done building, it will say “Waiting for bootloader” at which point you will plug in your board with the flashed bootloader. The uploader tool will then handle the rest of the process
10. If successful, when plugged in the board will now load the PX4 bootloader for a moment, and then launch into the PX4 Application. At that point it should be identifiable and configurable by QGroundControl.

To get PX4 on FCIV, with the H743:

This is probably where the next team will resume our work. We were able to get the bootloader to compile and flash by using the Pixracer Pro build target and tweaking a few clock settings. We chose that build target because it fairly closely matches our current FCIV. Ideally a future board will closely match the FMU-v6C. We didn't make any meaningful progress beyond that, and the work we did isn't fit to be described here. It seems to be a similar process, and [this page](#) might be a good place to start looking.

Why PX4 still doesn't work for us

We had the goal of making PX4 work, and we made good progress with the available FCII board. These were the unforeseen roadblocks we didn't have time to overcome before the semester ended:

1. SD card interface. FCII is wired to use the SDMMC2 hardware interface of the F777. It is routed correctly, and in theory should work fine. Unfortunately, the drivers that PX4 uses don't seem to be well implemented for using that specific port on that specific processor. You would think it would be a matter of changing SDMMC1 to SDMMC2, but it was not. We spent a long time trying to make a workaround but were completely unsuccessful in this. This matters because the board needs somewhere to save configuration settings, otherwise it can't fly. Some boards have a small flash memory chip for this, which might be worth implementing in a future version of the board, but ours doesn't have it.
2. IMU Drivers. FCII is the version of the board we had to work with, but FCII has an IMU that isn't supported by PX4. That issue was fixed on FCIII, but that board can't run PX4. We made an attempt at custom drivers for the IMU, but weren't successful. This is a dead-end problem and shouldn't be pursued, but rather fixed in hardware by making sure any future version of the board has an IMU that's on [the PX4 list of supported IMU modules](#). FCIII+ should already have this.

3. There's probably more we don't know about. We only found out about 1 and 2 after we figured out and solved the other bootloader, clock, and USB issues. It would be unwise to assume these are the last two obstacles to overcome.

QGroundControl

This is the companion desktop software that is used to configure a PX4 board for flight and load it with automated missions. It can be connected through USB, or through a SiK radio UART bridge.

PX4IO Coprocessor

This is an optional separate processor that PX4 reference designs have. It handles the lowest level functions like sending PWM and reading RC controls and communicates with the main processor. This is for redundancy, so if the main processor hard faults on one of the tasks it is running, the coprocessor can give manual control of the drone back to the operator instead of the whole thing falling out of the sky. Our design doesn't have one of these coprocessors and should be explicitly disabled when configuring the PX4 build.

Disable VBUS Sense

Due to the way the hardware was configured, in order for the USB connection to work at all we needed to add the flag "BOARD_USB_VBUS_SENSE_DISABLED 1" to the code. How to implement this can be found in Appendix A. Without this, the USB connection would show no signs of life at all. With this, the computer could tell something was connected, but it wouldn't enumerate correctly, which we solved by avoiding overdrive mode.

STM32F777 Overdrive Mode

One thing that impeded progress for a long time is overdrive mode, or rather that we didn't know it existed. We couldn't get the USB to enumerate while running the PX4 Application and we didn't know why, but the signs kept pointing to clock issues. The clock speed of the F777 is 216MHz, but what we didn't know is that you have to enable something called overdrive mode to run it that fast. Usually, the STM software would handle this for you, but PX4 doesn't. After learning about this, we were able to reconfigure the processor to run at 168MHz, which is outside the overdrive mode range. This made our clock issues go away and the USB worked after that. This might not be relevant for the newer H7 processors but is worth noting.

Recommendations

FCII: Interface microSD with PX4 Autopilot®. This would let you save an airframe in QGROUNDCONTROL® to open peripheral configuration menus.

FCIV: Derive from FMUv6C board, a custom flight controller supported by PX4 Autopilot® and a close match to the Pixhawk® 6C Mini™.

FCV: Design hardware based on a simple PX4 template board. USE THE EXACT PIN CONNECTIONS.

See the FCIV Final Presentation; it gives a comprehensive view of the progress we made and the next steps to take.

Engineering Standards (ABET)

We followed the exact same engineering standards as in previous semesters. For further information on these standards, refer to the reports of previous semesters. These can be found in /Documentation/Reports under these filenames:

FinalReport_ECEN499_S25_G3.docx

FinalReport_ECEN499_W26.docx

Design to Meet Specific Needs (ABET)

We followed the exact same Design to Meet Specific Needs as in previous semesters. For further information on these needs, refer to the reports of previous semesters. These can be found in /Documentation/Reports under these filenames:

FinalReport_ECEN499_S25_G3.docx

FinalReport_ECEN499_W26.docx

Appendix A: PX4 Repository Modifications for using FCII

This appendix is regarding the modifications needed to add FCII as a build target for the PX4 bootloader and autopilot repositories. As mentioned above, any mention of FCIII is actually a reference to FCII, and was simply a typo that got propagated. Rather than provide code files to copy over, we are including instructions so they can't get separated from this document, and so the next team can learn what we're doing. Also in case the underlying code gets updated, a future team won't be using something old.

PX4-Bootloader

The build targets are defined in the hw_config.h file. Each board has its own block to define some basic things about it so the bootloader can build correctly. This is for the

F777 processor, and is not applicable to the H743. We added the following section to that file:

```
#elif defined(TARGET_HW_BYUI_FCIII)

# define APP_LOAD_ADDRESS          0x08008000
# define BOOTLOADER_DELAY          5000
# define INTERFACE_USB              1
# define INTERFACE_USART            1
# define USBDEVICESTRING            "PX4 BL BYUI FCIII.x"
# define USBPRODUCTID               0x0087
# define BOOT_DELAY_ADDRESS         0x00000200

# define BOARD_TYPE                  67
# define _FLASH_KBYTES               (*(uint16_t
*)0x1fff0f442)
# define BOARD_FLASH_SECTORS         11
# define BOARD_FLASH_SIZE             (_FLASH_KBYTES * 1024)

# define OSC_FREQ                     4

# define BOARD_PIN_LED_ACTIVITY       GPIO2
# define BOARD_PIN_LED_BOOTLOADER    GPIO2
# define BOARD_PORT_LEDS              GPIOE
# define BOARD_CLOCK_LEDS             RCC_AHB1ENR_GPIOEEN
# define BOARD_LED_ON                  gpio_set
# define BOARD_LED_OFF                 gpio_clear

# define BOARD_USART                  USART2
# define BOARD_USART_CLOCK_REGISTER   RCC_APB1ENR
# define BOARD_USART_CLOCK_BIT        RCC_APB1ENR_USART2EN

# define BOARD_PORT_USART             GPIOA
# define BOARD_PORT_USART_AF          GPIO_AF7
# define BOARD_PIN_TX                  GPIO2
# define BOARD_PIN_RX                  GPIO3
# define BOARD_USART_PIN_CLOCK_REGISTER RCC_AHB1ENR
# define BOARD_USART_PIN_CLOCK_BIT     RCC_AHB1ENR_GPIOAEN
# define SERIAL_BREAK_DETECT_DISABLED 1
# define BOARD_USB_VBUS_SENSE_DISABLED 1
```

Then you need to add a line referencing this new target to the board_types.txt file:

```
TARGET_HW_BYUI_FCIII          67
```

(This number should be the same as whatever is in "# define BOARD_TYPE" in hw_config.h, and shouldn't be the same as any of the other options in board_types.txt)

Next, in the main_f7 file, we changed line 148 to this:

```
.systick_mhz    = 216
```

We recognize now that 216 might not be the right number because of overdrive mode, but this worked so we didn't try to change it.

In the Makefile, we added this section to the build targets:

```
byui_fciii_bl: $(MAKEFILE_LIST) $(LIBOPENCM3)
    ${MAKE} ${MKFLAGS} -f Makefile.f7 TARGET_HW=BYUI_FCIII
    LINKER_FILE=stm32f7.ld TARGET_FILE_NAME=$@
```

(if it complains about spacing while trying to compile, make sure there's a tab before the line and not spaces, it's whitespace sensitive)

In the stm32f7.ld linker script, we changed the section on line 37 to be this:

```
MEMORY
{
    rom (rx)  : ORIGIN = 0x08000000, LENGTH = 32K
    ram (rwx) : ORIGIN = 0x20000000, LENGTH = 512K
}
```

And on line 111, this:

```
PROVIDE(_stack = 0x20080000);
```

(These are configuring the correct memory addresses and sizes on the F777)

PX4-Autopilot

We cloned the CubePilot CubeYellow board files because it has a very similar hardware configuration to ours. This section details how we set up a new folder for FCII.

1. Copy "boards/cubepilot/cubeyellow/" to "boards/yourname/yourboard/" and rename name and board to whatever you want. We used "byui/fciii/"
2. in /name/board/nuttx-config/nsh/defconfig, on line 56, change CONFIG_ARCH_BOARD_CUSTOM_DIR="../../boards/name/board/nuttx-config" to have your new name and board.
3. Change "board_id": in firmware.prototype to whatever "# define BOARD_TYPE" is set to in hw_config.h in the previous section.

4. Probably not necessary, but in `/boards/byui/fciii/firmware.prototype`, we changed the description and summary just to make them unique.

Now you should be able to use the command “make byui_fciii_default” to compile.

To make the memory addresses line up, we changed lines 74 and 75 of `byui/fciii/nuttx_config/scripts/script.ld` to this:

```
FLASH_ITCM (rx) : ORIGIN = 0x00208000, LENGTH = 2016K  
FLASH_AXIM (rx) : ORIGIN = 0x08008000, LENGTH = 2016K
```

Line 78 to this:

```
DTCM_RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 512K
```

and line 79 to this (the 2 changed to an 8):

```
SRAM1      (rwx) : ORIGIN = 0x20080000, LENGTH = 368K
```